

ImpactJobs Project Guide

Beginner-friendly build and handover document for a free NGO/CSR/development-sector job portal.

Project goal

Build a public job-search website that reads job records from Google Sheets, shows job cards on the homepage, and opens a complete job-detail page. The project must work with free hosting and the user's own domain.

Final architecture

Google Sheets → Google Apps Script Web App → Static HTML website → Cloudflare Pages → Custom domain

Current known status

A Google Sheet named Jobs exists. An Apps Script endpoint has already returned JSON containing job-001, Python Developer, and ABC Company. A local job-detail page works. A Cloudflare address exists, but the hosted homepage previously showed sample data because the Apps Script URL placeholder had not been replaced or the updated file was not deployed.

1. Required accounts and files

Accounts: Google account, GitHub account, Cloudflare account, and an owned domain. No separate API key is required for the public Apps Script read-only endpoint.

Required repository files:

```
job-website/  
■■■ index.html homepage  
■■■ job.html job-detail page  
■■■ style.css optional if CSS is separate  
■■■ script.js optional if JavaScript is separate
```

The current homepage may contain CSS and JavaScript inside index.html. That is acceptable for the first working version.

2. Google Sheet specification

Create a tab named Jobs. Row 1 must contain these exact headings:

```
ID | Title | Company | Company Logo | Job Description | Required Skills | What We Offer | Job  
Type | Urgency | Location | Date Posted | Expiration Date | Salary | Company Founded | Company  
Phone | Company Email | Company Website | Company Address | Apply URL | Map URL | Source | Status
```

Each later row is one job. ID must be unique. Status should be Published for public jobs and Hidden for jobs that must not appear.

Important data rule: use normal URLs, not Markdown links. Correct: <https://example.com>. Incorrect: [\(https://example.com\)](https://example.com).

3. Apps Script API

Open the Sheet → Extensions → Apps Script. The script must read the Jobs tab and return JSON from doGet().

```
const SHEET_NAME = "Jobs";

function doGet() {
  const sheet = SpreadsheetApp.getActiveSpreadsheet().getSheetByName(SHEET_NAME);
  const values = sheet.getDataRange().getDisplayValues();
  if (values.length < 2) return sendJson([]);
  const headers = values.shift();
  const jobs = values.filter(row => row[0] && row[21] !== "Hidden").map(row => {
    const job = {};
    headers.forEach((header, index) => job[header] = row[index]);
    return job;
  });
  return sendJson(jobs);
}

function sendJson(data) {
  return ContentService.createTextOutput(JSON.stringify(data))
    .setMimeType(ContentService.MimeType.JSON);
}
```

Deploy: Deploy → New deployment → Web app. Execute as Me. Who has access: Anyone. Copy the URL ending in /exec. Do not use /dev.

Test the URL directly in a browser. It must show JSON before connecting the website.

4. Connect the homepage

In index.html find:

```
const API_URL = "PASTE_YOUR_WEB_APP_URL_HERE";
```

Replace only the placeholder with the real Apps Script URL:

```
const API_URL = "https://script.google.com/macros/s/YOUR_ID/exec";
```

Keep quotation marks. Do not paste a Google Sheet URL, Apps Script editor URL, password, or Markdown-formatted link.

The homepage should call `fetch(API_URL)`, render returned jobs, and use sample jobs only as a fallback. For production troubleshooting, temporarily remove the fallback so errors are visible.

5. Job detail page

Use one reusable page: `job.html?id=job-001`. The page reads the ID, fetches the same JSON endpoint, finds the matching job, and displays title, company, description, skills, offer, location, dates, salary, company details, and Apply URL.

```
const params = new URLSearchParams(window.location.search);  
const jobId = params.get("id");  
const job = jobs.find(item => String(item.ID) === String(jobId));
```

Homepage links must use `job.html?id=THE_SHEET_ID`. The ID in the link must exactly match the Sheet ID.

6. Local testing checklist

1. Open Apps Script /exec URL: JSON appears. 2. Put the real /exec URL in index.html. 3. Open index.html locally: real Sheet job appears. 4. Click View Job: job.html?id=job-001 opens. 5. Click Apply Now: Apply URL opens. 6. Add a second Sheet row and confirm it appears after refreshing.

If sample Python Developer appears, the homepage is using the fallback. Check the API_URL line, save the file, upload the file, and hard-refresh with Ctrl+F5.

If “Could not load jobs” appears, check: endpoint is /exec; deployment is Anyone; endpoint opens in an incognito window; browser console has no URL error; file was actually redeployed.

7. GitHub deployment

Create a public repository named job-website. Upload index.html, job.html, style.css, and script.js if present. Commit changes. Confirm the latest index.html on GitHub contains the real API URL, not the placeholder.

8. Cloudflare deployment

Workers & Pages → Create application → Pages → Connect to Git → select repository. Framework: None. Build command: blank. Output directory: /. Deploy. Open the generated pages.dev or workers.dev address.

After a GitHub change, wait for a new deployment. Hard-refresh the public site. Use the deployment log if the old content remains.

9. Custom domain

In Cloudflare project → Custom domains → Set up a domain. Follow the displayed DNS instructions. Do not change DNS until the site works on the free Cloudflare address.

10. Data and duplicate plan

For each source, create a stable job ID. Prefer the original job URL as the unique key. Before appending a job, compare its URL or a normalized combination of title, company, location, and URL. Do not publish duplicates.

Suggested future automation order: first manual Sheet rows, then one official API or RSS feed, then duplicate checking, then a second source. Avoid scraping login-protected or blocked websites and follow each source's terms.

11. Codex handover prompt

Give Codex the following project instruction:

```
You are maintaining ImpactJobs, a static HTML job portal.  
Data source: Google Sheet tab Jobs exposed by a Google Apps Script doGet JSON endpoint.  
Frontend: index.html homepage and job.html detail page.  
Hosting: Cloudflare Pages or Workers static deployment.  
Never replace the Apps Script URL with a Sheet URL. Keep the endpoint in one clearly marked  
API_URL constant.  
Use exact Sheet headings and preserve job IDs.  
Homepage must show published jobs, search, filters, responsive layout, and links job.html?id=ID.  
Detail page must load the matching ID and safely escape user-provided HTML.  
Do not add paid services or API keys.  
When changing code, explain which file changed, preserve existing design, and provide a manual  
test checklist.
```

12. Acceptance criteria

The project is complete when: the public homepage loads without login; real Sheet jobs appear; search works; filters do not crash; each job opens its detail page; Apply Now opens the Sheet Apply URL; hidden rows are not public; mobile layout works; a new Sheet row appears after refresh; the custom domain loads HTTPS; no private credentials are committed to GitHub.

13. Do not do these things

Do not put a Google password or private API key in HTML. Do not use /dev in production. Do not expose private Sheet columns. Do not assume a job website permits scraping. Do not overwrite working files without keeping a backup. Do not add automatic posting or deletion until the read-only display is stable.